

DCForge

Solution-as-Code 客户解决方案编译与价值验证引擎

比赛项目亮点与创新性说明

核心主张

不是让大模型“写一份方案”，而是把客户解决方案变成
可编译、可校验、可重编译、可执行的 Solution-as-Code。

15	3	7	6	161
能力胶囊	方案档位	业务场景	约束类型	自动化测试

项目阶段：Solution 模块已完成；Runtime 与前端 Demo 按团队计划推进

版本依据：项目 README、测试报告与 A→B 联调记录 • 2026 年 7 月

一、项目亮点概览

DCForge 面向企业数字化方案设计中的“需求不结构化、方案依赖经验、约束容易遗漏、变更难追踪、价值难验证”等问题，构建了一套以公共合同为边界、以确定性工具链为核心、以大模型 Agent 为调度层的解决方案编译系统。

一句话亮点

DCForge 把客户需求从“文本描述”升级为“可编译的工程对象”，让方案生成从一次性内容创作转变为可验证、可追踪、可迭代的软件工程过程。

1.1 评委应关注的六个创新点

创新方向	项目实现
Solution-as-Code	用 ProcessSpec、SolutionBundle、RunReport 三类公共合同贯通需求、方案和执行，使解决方案具备版本、结构和执行语义。
Agent + 确定性工具	大模型只做意图识别、约束结构化、工具选择和解释；核心业务结果由确定性 Python 工具生成。
三档方案编译	同一需求一次生成 conservative、balanced、innovative 三套差异化方案，支持比较而非单点答案。
增量重编译	客户新增安全、审批或数据约束后，不推倒重做，而是输出组件 Diff、节点 Diff 与变更原因。
跨场景自适应	根据行业、部门、痛点、数据与目标识别 7 类场景，组件、流程、required_data 均随场景变化。
人机协同安全边界	所有 human-approval 节点统一 human_gate=true，为 Runtime 中的暂停、审批和审计提供明确语义。

1.2 当前可验证成果

10/10	5	30/30	5	0
A 数据编译成功	场景化组件集合	人工审批门有效	REST API	非采购硬编码违规

上述数字是当前版本的工程验证结果，不等同于真实企业生产环境中的业务收益。项目明确区分“设计阶段评分”和“运行阶段价值指标”，避免以 Demo 数据替代真实效果。

二、项目定位：从“生成方案”到“编译方案”

2.1 传统方案设计的结构性痛点

- 客户需求通常以会议纪要、口述、表单或文档存在，信息缺失和表达歧义难以被系统直接消费。
- 顾问常依靠个人经验拼接产品能力，方案之间难以复用、比较和复现。
- 客户新增“数据不得出域”“高风险必须人工复核”等条件后，传统方式往往需要人工重写整份方案。
- 方案评审通常依赖主观判断，缺少硬约束校验、结构检查和清晰的评分维度。
- 方案展示与后续执行脱节，文档中的流程节点没有明确的机器执行和人工审批语义。

2.2 DCForge 的核心范式

项目将企业解决方案抽象为三层结构化对象：

阶段	公共合同	作用
需求阶段	ProcessSpec	表达行业、部门、目标、现状流程、痛点、可用数据、系统、约束、目标指标及缺失信息。
方案阶段	SolutionBundle	保存同一项目的三套 SolutionPlan，包括组件、目标流程、实施步骤、假设、告警与评审分数。
执行阶段	RuntimeRequest / RunReport	承接选定方案，记录节点执行、人工审批、审计事件与价值指标。

关键创新

“合同优先”使 A、B、C 三个模块不需要共享内部 Prompt 或实现细节，只需遵循稳定的数据对象即可联调。

2.3 与普通 LLM 方案生成的本质区别

比较维度	普通文本生成	DCForge
输出形态	一段自然语言方案	严格 Pydantic 校验的 SolutionBundle
稳定性	相同输入可能大幅波动	核心结果由确定性工具生成，可复现
约束处理	依赖 Prompt 提醒	硬约束进入结构化校验器
变更处理	重新生成整份文本	增量重编译并输出 Diff
可解释性	主要依赖模型自述	工具轨迹、评分维度、组件原因、变更原因
执行衔接	文档与执行分离	to_be_nodes 与 human_gate 面向 Runtime

三、技术架构创新：Agent 负责决策，工具负责事实

DCForge 不让大模型直接写入关键业务字段，而是将其限定在安全、可控的调度层。Solution Agent 识别 compile、review、recompile、explain 等意图，再调用白名单工具。

DCForge：从需求到可执行方案的编译链路



3.1 大模型与确定性工具的职责边界

由 Qwen Agent 负责	由确定性 Python 工具负责
识别用户意图	生成 SolutionBundle 与三套 SolutionPlan
把自然语言约束结构化为 BusinessConstraint	计算 review_score 与五个评审维度
选择并调度允许的工具	选择 selected_components 与 required_data
解释方案差异、评分与限制	生成 to_be_nodes、human_gate 与 gate_reason
在有限步数内组织回答	生成 RecompileResult、组件 Diff 与节点 Diff

3.2 为什么这种架构更适合企业场景

- 可控：关键字段不能被模型任意覆盖，工具白名单禁止 shell、eval、文件写入、Git、数据库和网络搜索。
- 可审计：每次 Agent 执行记录工具名称、步骤、状态和摘要，可回看系统如何得出结论。
- 可回退：模型不可用时，compile 场景仍可使用确定性编译链路，避免核心服务完全失效。
- 可测试：自动测试使用 FakeLLMProvider，不消耗真实 API 额度，也不把网络波动带入 CI。
- 可替换：LLMProvider 使用 OpenAI-compatible 抽象，可在不改业务工具的前提下替换模型服务。

四、方案编译创新：一份需求，生成三种可比较的工程方案

4.1 三档方案不是换文案，而是换工程取舍

方案档位	设计取向	典型特征	适用情形
conservative	风险优先	组件较少、人工控制较强、实施复杂度较低	风险敏感、快速落地、系统基础薄弱
balanced	综合权衡	保留关键人工审批，在自动化、成本和风险之间平衡	大多数常规企业改造项目
innovative	能力优先	引入更多 RAG、本地模型、监控、质量看板和反馈闭环	追求扩展性、允许更高实施复杂度

项目不强制 balanced 必须得分最高。Reviewer 可根据可行性和风险让 conservative 获得更高设计分；balanced 的推荐语义是“综合权衡”，而不是“绝对最优”。

4.2 能力胶囊：将产品能力变成可组合构件

系统维护 15 个能力胶囊，覆盖文档抽取、企业 RAG、字段完整性检查、规则引擎、异常分类、风险评分、本地模型、数据脱敏、人工审批、工单路由、通知、审计、质量看板、流程监控和反馈闭环。每个胶囊带有组件 ID、执行类型、所需数据和适用条件。

创新价值

能力胶囊让“产品清单”变成“编译组件库”。检索器不是搜索相似文本，而是根据 ProcessSpec 的行业、部门、痛点、数据和约束进行稳定选择。

4.3 多维 Reviewer：把“看起来不错”变成结构化评审

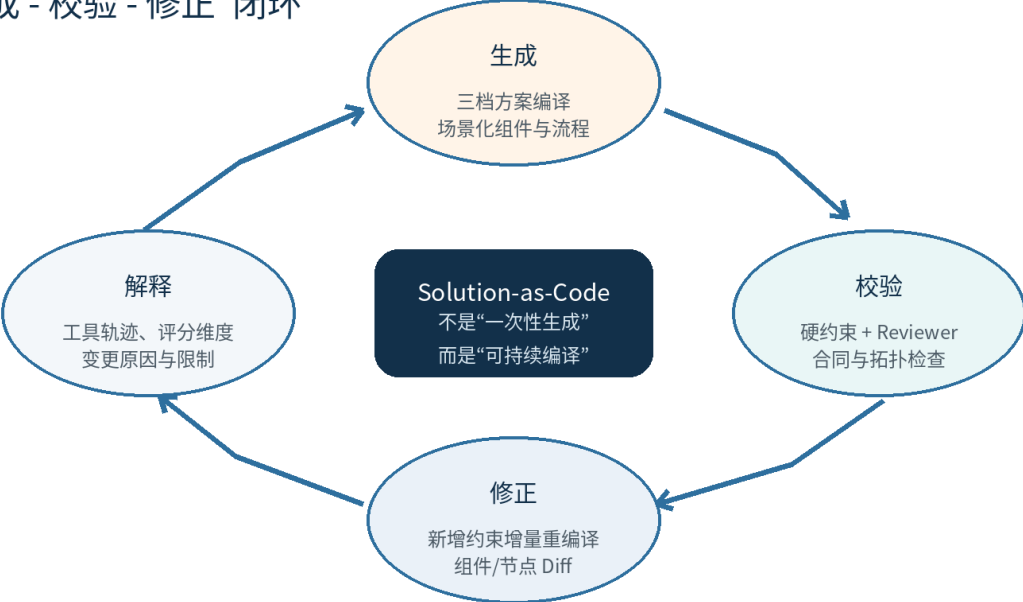
维度	权重	评审内容
约束合规	30	硬约束是否满足，是否存在不可验证项或冲突。
需求覆盖	25	方案组件和流程是否覆盖核心痛点、业务目标与数据条件。
工作流完整性	20	节点、边、执行者、人工门和审计链是否完整。
可解释性与证据	15	组件原因、假设、告警、证据和变更说明是否充分。
可行性	10	实施复杂度、依赖条件和当前信息完整度是否可接受。

review_score 是设计阶段评分，用于方案比较和发现结构问题，不代表真实生产环境中的业务效果。

五、核心创新：客户约束变化后的增量重编译

企业项目的常态不是“一次定稿”，而是客户在评审过程中不断追加条件。DCForge 将这种变化视为重新编译，而不是重新写文档。

“生成 - 校验 - 修正”闭环



5.1 RecompileResult 的可解释 Diff

输出字段	含义	评委可观察的价值
previous_solution_id	上一版本方案标识	明确版本来源，避免方案混淆
new_solution.solution_id	新版本方案标识	例如 balanced-v1 → balanced-v2
added_component_ids	新增能力组件	说明新约束带来的能力补充
removed_component_ids	移除能力组件	说明不再适用或产生冲突的能力
changed_node_ids	受影响流程节点	定位流程变化范围
change_explanations	逐项变更原因	让客户知道为什么改、改了什么

5.2 已验证的重编译示例

示例约束

客户新增：“敏感数据不得出域，必须脱敏、本地处理并记录审计日志。”

真实 qwen3.7-plus Agent 将自然语言整理为结构化安全约束；确定性 recompile_solution 对 balanced-v1 进行重编译，生成 balanced-v2，并新增 data-masking 与 local-model 等组件，同时输出受影响节点和解释。

- 创新点一：约束不是附加在文档末尾，而是进入结构化编译输入。
- 创新点二：方案版本、组件差异和流程差异可机器读取。
- 创新点三：大模型不直接伪造 Diff，变更结果来自确定性重编译器。

六、跨场景自适应：避免“一套采购模板打天下”

项目早期编译器以采购异常为主。通过 B-M7 增强，系统增加确定性场景识别层，不使用 project_id，而是分析 industry、department、business_goal、available_data、pain_points 和 target_metrics。

场景类型	代表业务	方案组件与流程特征
incident_response	网络故障、系统中断、服务不可用	故障分类、工单路由、监控、状态通知、人工确认、审计
fraud_risk	欺诈、盗刷、未授权交易	异常分类、风险评分、规则引擎、脱敏、高风险人工复核
identity_account	姓名或账户资料变更、身份核验	材料提取、字段完整性检查、身份规则、脱敏、人工审批
dispute_investigation	征信争议、调查复核、申诉	材料提取、企业 RAG、调查工单、人工复核、审计
customer_service	客户咨询、投诉、问题分流	知识检索、异常分类、工单路由、通知、反馈闭环
procurement_exception	采购订单、发票、收货与供应商异常	单据抽取、规则核验、风险评分、审批、审计
generic	无法明确分类的业务流程	字段检查、规则引擎、人工复核、工单路由、审计

6.1 场景化不仅改变节点名称

- 组件集合变化：不同场景选取不同能力组合，而不是仅修改摘要。
- 工作流变化：故障处置、欺诈复核、身份变更和争议调查拥有不同节点序列。
- required_data 变化：网络故障使用告警记录和客服工单；欺诈场景使用交易记录和风险特征；身份场景使用证明材料和变更申请。
- 审批原因变化：人工门的 gate_reason 根据场景生成，不再固定使用“超过 50 万元”等采购措辞。

工程验证

10 条 A 模块 ProcessSpec 全部编译成功，形成 5 种组件集合和 5 种节点序列；非采购场景全字段采购术语违规为 0。

七、人机协同与安全创新：把人工审批写进方案语义

7.1 human_gate 是执行语义，不是展示标签

当组件 ID 为 human-approval 时，系统统一要求 executor="human"、human_gate=true，并生成非空 gate_reason。这意味着下游 Runtime 不是“显示一个人工节点”，而是必须在该节点暂停、等待批准或拒绝，再决定继续或终止。

字段	执行含义
executor = human	该节点由人工角色完成，不能伪装成自动执行。
human_gate = true	Runtime 到达该节点必须暂停，不能继续自动流转。
gate_reason	解释为何需要人工介入，如高风险交易复核或大范围故障确认。
next_ids	批准后继续流向后继节点；拒绝后的处置由 Runtime 业务规则决定。

7.2 Agent 安全边界

安全措施	实现方式
密钥隔离	LLM_API_KEY 仅从环境变量读取，.env 不进入仓库。
工具白名单	Agent 只能调用 compile、review、recompile、validate、retrieve 五类工具。
禁止高风险能力	禁止 shell、eval、任意文件读写、Git、数据库和网络搜索。
最大步数	Agent 最多执行固定步骤，防止无限循环和重复调用。
错误脱敏	Provider 处理 401/403/429/5xx/超时，错误信息不包含 API Key。
测试隔离	自动测试注入 FakeLLMProvider，不调用真实付费 API。
合同拒绝额外字段	公共 Pydantic 模型使用 extra="forbid"，防止静默接受未知数据。

创新价值

DCForge 将“人类在环”从产品口号变成数据合同和执行控制点，为高风险业务提供清晰的自动化边界。

八、工程可信度：以测试、合同和真实 API 验证支撑创新

8.1 多层验证体系

验证层	验证内容	当前结果
公共合同	ProcessSpec、SolutionBundle、Runtime 合同字段与 extra=forbid	3 项合同测试通过
能力与检索	15 个胶囊的数据完整性与确定性检索	模型、数据、排序测试通过
编译器	三套方案、稳定 ID、节点边、组件引用、场景自适应	跨场景与结构测试通过
约束与评审	6 类约束、5 维 Reviewer、不可验证项处理	规则与集成测试通过
增量重编译	版本递增、组件/节点 Diff、变更说明	重编译测试通过
Agent 与 Provider	意图识别、工具轨迹、失败回退、密钥保护	Fake Provider + API 测试通过
真实模型	阿里云百炼 qwen3.7-plus OpenAI-compatible API	compile 与 recompile 冒烟通过
A→B 联调	10 条 ProcessSpec 批量编译与场景适配	10/10 成功

8.2 当前版本关键数字

161	10/10	3	30/30	5
全量自动化测试	需求编译成功	每项目方案数	审批门有效	真实 API 端点

8.3 真实性与口径控制

- review_score 是设计阶段结构评分，不是上线后的业务绩效。
- expected_metrics 是待验证指标名称，不代表已经取得改善。
- 10/10 编译成功说明合同与工具链可用，不代表所有行业规则已经穷尽。
- 真实 API 冒烟证明模型调用链可用，不代表模型在所有自然语言约束下均不会出错。
- 预算和周期约束因公共合同缺少精确成本/工期字段，当前会标记为不可验证，而不是编造结论。

九、比赛竞争力与差异化分析

9.1 与常见参赛方案的区别

常见方案形态	常见不足	DCForge 的差异化
聊天机器人 / 方案问答	输出停留在文本，难以验证和执行	输出严格 SolutionPlan、流程节点、组件与约束结果
RAG 知识助手	擅长检索，但缺少完整方案构建与版本变化	能力检索只是工具之一，后续还有编译、评审和重编译
workflow编排 Demo	流程预先写死，难以从客户需求自动生成	从 ProcessSpec 自动选择组件和生成三档流程
单 Agent 自主规划	灵活但稳定性、审计性和安全边界不足	Agent 只做决策，业务字段由确定性工具产生
静态数字化方案书	客户变更后依赖人工重写	通过 RecompileResult 输出可机器读取的差异

9.2 对评审维度的直接贡献

比赛评审维度	DCForge 对应亮点
创新性	提出 Solution-as-Code，把解决方案设计转化为可编译、可重编译的软件工程过程。
技术深度	结合 LLM Agent、Pydantic 合同、确定性检索、编译器、约束验证、Reviewer 和 Diff。
完整性	覆盖需求输入、方案生成、评审、变更和下游 Runtime 合同，而不是单一模型调用。
可解释性	工具轨迹、组件原因、评分维度、假设、告警和变更原因均可展示。
安全合规	硬约束校验、人工审批门、数据脱敏、本地模型与审计能力可进入方案结构。
扩展性	新增场景、能力胶囊或模型 Provider 时，不需要推翻公共合同和核心工具。
工程质量	SDD + TDD、161 项测试、真实模型冒烟、A→B 批量联调。

最适合答辩强调的观点

项目创新不在于“用了一个大模型”，而在于把大模型约束在可审计的编译流程中，并将企业方案设计的结构、约束、版本和执行语义统一起来。

十、应用价值、扩展方向与当前边界

10.1 潜在应用场景

应用方	使用方式	价值
数字化解决方案顾问	输入客户现状与约束，快速形成多档方案和评审材料	减少重复整理，提高方案一致性
售前与交付团队	把产品能力胶囊与客户痛点匹配，保留方案版本和变更记录	降低交接损耗，提高可追踪性
企业流程治理部门	将审批、安全、数据与审计要求前置到方案设计阶段	减少约束遗漏和返工
行业解决方案平台	扩展能力胶囊和场景规则，形成可复用方案市场	支持跨行业复制与组合
AI 应用治理团队	通过 Agent 工具白名单和确定性结果边界审查模型行为	增强大模型应用可控性

10.2 下一阶段路线

1. Process Agent：把自然语言、表单和文档进一步自动整理为 ProcessSpec，并显式提出澄清问题。
2. Runtime：执行选定 SolutionPlan，支持节点状态、human_gate 暂停、批准/拒绝和 RunReport。
3. 前端 Demo：展示三套方案差异、流程图、约束检查、Agent 工具轨迹和重编译 Diff。
4. 价值验证：将 expected_metrics 转为可计算的 before/after 指标，并明确数据来源与置信度。
5. 证据增强：为能力胶囊补充官方产品文档、行业标准和公开案例的 evidence_urls。
6. 分支/并行工作流：从当前线性链扩展到条件分支、并行节点和异常补偿。

10.3 当前边界与风险

边界	当前处理
Runtime 与前端尚未完成	README 明确标记为规划中，不将合同示例描述为已经运行。
工作流当前以线性链为主	已保证节点和边完整，分支并行留待后续扩展。
evidence_urls 暂为空	不伪造引用，后续补充公开证据。
预算与工期难以精确验证	公共合同缺少精确成本/周期字段时标记 unverifiable。
模型约束抽取可能出错	通过 Pydantic 校验、缺失信息提示和工具边界降低风险。
真实企业效果尚未验证	所有指标与评分均按设计阶段或 Demo 口径说明。

结语

DCForge 的核心价值是把企业解决方案从“依赖个人经验的静态文档”转化为“有合同、有组件、有流程、有约束、有评分、有版本、有执行语义”的工程资产。其创新性来自大模型 Agent 与确定性编译系统的协同：既保留自然语言交互的灵活性，又通过工具、合同、测试和人工审批边界保证稳定性与可信度。

答辩收束句

DCForge 不是再做一个会写方案的 Agent，而是在构建企业解决方案的“编译器与运行基础设施”。

附录：项目创新要点速查

编号	创新要点	可展示证据
01	Solution-as-Code 三层公共合同	ProcessSpec → SolutionBundle → RunReport
02	Qwen Agent 与确定性工具分工	AgentToolCall、Provider 注入、工具白名单
03	同一需求三档方案编译	conservative / balanced / innovative
04	六类硬约束与五维 Reviewer	constraint validation、review_score、warnings
05	客户新增约束后的增量重编译	v1→v2、added/removed/changed、change_explanations
06	七类业务场景自适应	scenario.py、组件集合与节点序列差异
07	场景化 required_data	非采购场景全字段采购术语违规为 0
08	人工审批执行语义	human-approval → executor=human → human_gate=true
09	真实大模型接口验证	阿里云百炼 qwen3.7-plus compile/recompile 冒烟
10	工程质量与安全	161 tests、Pydantic extra=forbid、密钥隔离、Fake Provider

DCForge

从静态方案文档，走向可编译的企业解决方案

项目创新总结

用公共合同承载需求与方案，用确定性工具保证结构与约束，用大模型 Agent 提供自然语言交互与调度能力，用增量重编译响应客户变化。

可编译

结构化方案

可校验

约束与评分

可演进

版本与 Diff

资料与数据口径

本材料依据当前项目 README、Solution 模块实现规格、自动化测试结果、真实模型冒烟记录及 A→B 联调报告整理。

测试数字和能力数量对应当前版本；设计阶段评分、预期指标和 Demo 数据不构成真实生产效果承诺。